

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

BUILDING A FILE SHARING APPLICATION

2nd Progress Report: System & Protocol Design

Người thực hiện:

Cao Van Dong
Luong Ngoc Vu Long
Tran Le Minh

ONE LOVE. ONE FUTURE.

1. Application Introduction & Problem Statement

Motivation & Collaboration Needs

- **Structured Team Workspaces**: Projects require dedicated, group-isolated folders to organize code, documents, and assets without cross-group leakage.
- **Preventing Accidental Loss**: Public/open shared drives suffer from unauthorized deletions and unintended overwrites by arbitrary members.
- **Network Fragility**: Incomplete network transfers leave corrupt, unusable ghost files on the server without transactional staging.

Proposed System: Group File Sharing

- **Client-Server Architecture**: Central server guarantees 24/7 availability and acts as authoritative access control enforcement.
- **Rigid RBAC Model**: Members can upload, download, and create subfolders; **only Group Admins** can rename or delete files and folders.
- **Resilient Framing & Staging**: Custom LTV protocol over TCP, bounded chunk streaming, and atomic commit staging with SHA-256 verification.

2. System Functionality & Use Case Model

System Actors & Scope

- **Guest (Unauthenticated)**: Registers account (`AUTH_REGISTER`) and signs in via credentials (`AUTH_LOGIN`).
- **Regular Group Member**: Joins groups, browses directory trees (`DIR_LIST`), uploads files, downloads assets, and creates subfolders.
- **Group Admin (Creator)**: Reviews join requests (`GROUP_JOIN_REVIEW`); holds exclusive authority to rename and delete files and folders.

Core Collaboration Use Cases

- **Identity & Session (UC1)**: Account registration, credential authentication, and persistent session binding.
- **Group Lifecycle (UC2–UC3)**: Create group (auto-assigns Admin), browse groups, and review pending member requests.
- **Workspace Navigation (UC4, UC7)**: Hierarchical subfolder navigation and directory creation.
- **Streamed File Transfer (UC5–UC6)**: Chunked upload/download with SHA-256 checksums and resume support.
- **Administrative Protection (UC8)**: Exclusive admin controls for folder/file deletion and renaming.
- **Real-Time Notifications (UC9)**: Server push events for member joins, uploads, and modifications.

Group-Scoped Security Policy

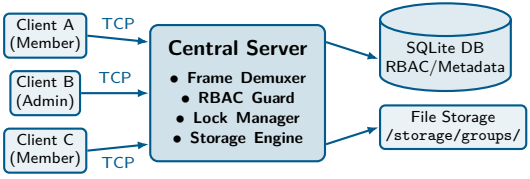
Permissions evaluated dynamically per command on (`userId`, `groupId`, `role`).

Operation	Guest	Non-Member	Member	Group Admin	Security Policy / Enforcement Rule
`AUTH_LOGIN` / `REGISTER`	✓	✓	✓	✓	Public unauthenticated commands
`GROUP_CREATE`	×	✓	✓	✓	Authenticated user becomes Admin of new group
`GROUP_JOIN_REQ`	×	✓	×	×	Non-members can submit pending request
`GROUP_JOIN_REVIEW`	×	×	×	✓	**Admin only**: Approve or reject applicants
`DIR_LIST` / `FILE_INFO`	×	×	✓	✓	Restricted to approved group members
`FILE_DOWNLOAD`	×	×	✓	✓	Read access granted to all members
`FILE_UPLOAD` / `DIR_CREATE`	×	×	✓	✓	Write access to contribute files / subfolders
`FILE_RENAME` / `DIR_RENAME`	×	×	×	✓	**Admin only**: Returns 403 `FORBIDDEN` otherwise
`FILE_DELETE` / `DIR_DELETE`	×	×	×	✓	**Admin only**: Returns 403 `FORBIDDEN` otherwise

Paradigm Evaluation

- **Pure P2P (Discarded)**: NAT/firewall traversal barriers; offline peers cause file unavailability; complex RBAC consensus.
- **Hybrid (Discarded)**: High signaling overhead; asymmetric bandwidth; weak consistency during renames.
- **Client-Server (Selected Architecture)**:
  - **Authoritative RBAC**: Single source of truth for access control.
  - **24/7 Availability**: Continuous centralized file access.
  - **Atomic Concurrency**: Server lock arbitration avoids races.

High-Level System Topology



5. Server Engine & Multi-Tier Architecture

Server Component Tiers

- **I/O Multiplexer Layer**: Persistent TCP sockets; worker pool decodes LTV frames asynchronously without blocking the network loop.
- **Session & RBAC Interceptor**: Active session table; validates role permissions prior to command dispatch.
- **Concurrency & Lock Manager**: Path-scoped `ReadWriteLock`; multiple concurrent readers vs single writer.
- **Dual Storage Layer**: Relational SQLite (accounts, RBAC, metadata) + partitioned file tree.

Server Processing Pipeline

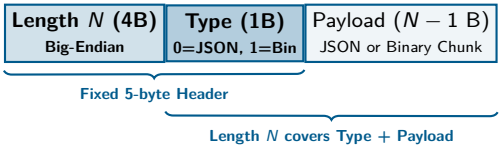
1. **TCP Stream Demuxer**: Accumulates bytes; verifies 5-byte header; extracts frame
2. **Authentication & RBAC**: Validates session token and role permissions
3. **Lock Arbitration**: Acquires ReadLock (Read/List) or WriteLock (Delete/Commit)
4. **Worker Pool Execution**: Queries DB / streams chunks / commits file
5. **Outgoing Response & Push**: Replies to caller; broadcasts async change events

6. Protocol Design: Transport Choice & TCP Framing

Transport Layer: TCP Byte-Stream

- **Why TCP**: Demands 100% byte reliability, in-order delivery, and flow control for file transfers.
- **Boundary Challenge**: Byte streams lack message boundaries; leads to packet fragmentation and concatenation across socket reads.
- **Solution: LTV Framing**: 5-byte header prefix defines frame length; receiver buffers until full frame arrives.

Length-Type-Value (LTV) Frame Layout



- $N = 1 +$ Payload size in bytes.
- **Type 0x00**: UTF-8 JSON Control / Event Message.
- **Type 0x01**: Raw Binary File Chunk with offset header.

7. Framing Specification: Type 0x00 vs Type 0x01

Type 0x00: JSON Control / Event Frame

Used for authentication, directory navigation, transfer control, and push event broadcasting.

Length N (4B)	**Type = 0x00 (1B)**	UTF-8 JSON Payload (Request / Response / Event envelope)

Type 0x01: Raw Binary File Chunk Frame

Used for high-speed file chunk streaming without base64 overhead (33% bandwidth savings).

Length (4B)	**Type = 0x01**	Transfer UUID (16B)	Offset (8B)	ChunkLen (4B)	**Raw File Bytes (ChunkLen)**

- **Transfer UUID (16 Bytes)**: Identifies the active upload/download staging session.
- **Offset (8 Bytes)**: Validates sequential chunk order and enables resumption after interruption.
- **ChunkLen (4 Bytes)**: Payload size of the raw byte slice (standard chunk: 32 KiB / 64 KiB).

8. Message Envelope Schema

Client Request Envelope

```
{
  "v": 1, "kind": "REQUEST",
  "reqId": "c8f2b740-9b3a-4f51-b8ef-101",
  "cmd": "FILE_UPLOAD_INIT",
  "payload": {
    "groupId": 4, "path": "/reports/final.pdf",
    "fileSize": 1048576, "sha256": "e3b0c442..."
  }
}
```

Server Push Event (Unsolicited)

```
{
  "v": 1, "kind": "EVENT",
  "event": "GROUP_FILE_ADDED",
  "payload": {
    "groupId": 4, "path": "/reports/final.pdf",
    "byUser": "dongcv"
  }
}
```

Server Response Envelope

```
// Successful Response
{
  "v": 1, "kind": "RESPONSE",
  "reqId": "c8f2b740-9b3a-4f51-b8ef-101",
  "cmd": "FILE_UPLOAD_INIT",
  "ok": true,
  "payload": {
    "transferId": "9b1deb4d-3b7d-4bad...",
    "chunkSize": 32768, "nextOffset": 0
  }
}
// Error Response
{
  "v": 1, "kind": "RESPONSE",
  "reqId": "d91a1002-8a1c-4033-9112-202",
  "cmd": "DIR_DELETE",
  "ok": false,
  "error": {
    "code": "FORBIDDEN_NOT_ADMIN",
    "message": "Only Group Admin can delete."
  }
}
```

Authentication & Group Membership Protocol Commands

Command	Caller Role	Key Request Parameters	Key Response / Event Payload
`AUTH_REGISTER`	Unauthenticated	`username`, `passwordHash`, `fullName`	`userId`, status: `"REGISTERED"`
`AUTH_LOGIN`	Unauthenticated	`username`, `passwordHash`	`userId`, `sessionToken`, `joinedGroups`
`AUTH_LOGOUT`	Authenticated	None	Invalidates session, releases locks
`HEARTBEAT_PING`	Any connected	`timestamp`	`pong`, connection health check
`GROUP_CREATE`	Authenticated	`groupName`, `description`	`groupId`, creator becomes **Group Admin**
`GROUP_LIST`	Authenticated	Filter: `"ALL"` or `"MY_GROUPS"`	Array of `[groupId, name, role, adminName]`
`GROUP_JOIN_REQ`	Non-Member	`groupId`, optional `message`	`reqId`, status: `"PENDING"`; notifies Admin
`GROUP_JOIN_REVIEW`	**Group Admin**	`reqId`, `action: "ACCEPT"|"REJECT"`	Updates role to `MEMBER`; triggers `EVENT`
`GROUP_MEMBERS`	Group Member	`groupId`	Roster of members with roles (`MEMBER`/`ADMIN`)
`GROUP_LEAVE`	Group Member	`groupId`	Revokes access; Admin cannot abandon group

Directory Structure & File Metadata Operations

Command	Caller Role	Key Request Parameters	Key Response / Error Behavior
DIR_LIST	Member / Admin	groupId, dirPath	Entries: [name, isDir, size, modified, author]
DIR_CREATE	Member / Admin	groupId, parentPath, dirName	Creates subfolder; broadcasts EVENT_DIR_CREATED
DIR_RENAME	**Admin Only**	groupId, path, newName	Renames folder; rejects members with FORBIDDEN
DIR_DELETE	**Admin Only**	groupId, path, recursive	Recursively deletes directory; checks write lock
FILE_INFO	Member / Admin	groupId, filePath	Returns size, SHA-256 hash, author, timestamp
FILE_RENAME	**Admin Only**	groupId, filePath, newFilename	Renames file; rejects members with FORBIDDEN
FILE_DELETE	**Admin Only**	groupId, filePath	Removes file; broadcasts EVENT_FILE_DELETED

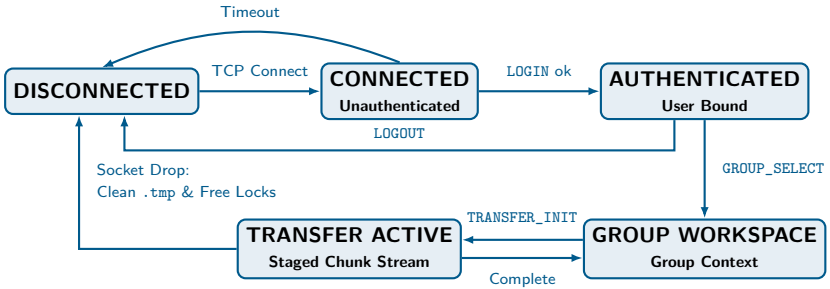
11. Command Catalog: File Transfer Lifecycle

Staged Upload, Resilient Streaming, and Download Commands

Command	Caller Role	Key Request Parameters	Key Response / Protocol Behavior
`FILE_UPLOAD_INIT`	Member / Admin	`groupId`, `path`, `fileSize`, `sha256`	Allocates `.tmp` file; returns `transferId`, `chunkSize`
`Type 0x01 Chunk`	Member / Admin	`UUID`, `offset`, `chunkLen`, `rawBytes`	Ingests binary chunk; advances staging offset
`FILE_UPLOAD_FINISH`	Member / Admin	`transferId`, `finalSha256`	Verifies SHA-256; atomic rename to repository
`FILE_UPLOAD_ABORT`	Member / Admin	`transferId`, `reason`	Deletes temporary `.tmp` staging file immediately
`FILE_DOWNLOAD_INIT`	Member / Admin	`groupId`, `filePath`, `offset`	Acquires ReadLock; returns `transferId`, `totalChunks`
`Type 0x01 Stream`	Server → Client	`UUID`, `offset`, `chunkLen`, `rawBytes`	Server streams sequential chunks to client socket
`FILE_DOWNLOAD_ACK`	Member / Admin	`transferId`, `status: "COMPLETE"`	Releases ReadLock; finishes transfer session

Connection Lifecycle States & Event Transitions

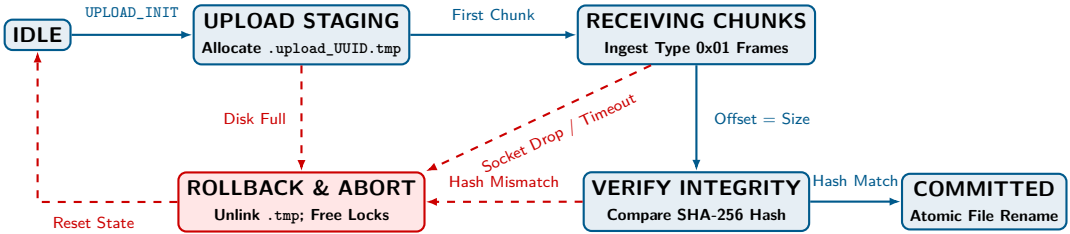
Guarantees session boundaries and timely reclamation of server resources.



- **Socket Drop Guard**: Socket disconnect at any stage triggers instant release of active Read/Write locks and unlinks incomplete staging files.

Staged File Upload Lifecycle: Preventing Corrupted Transfers

Directly resolves the challenge of broken TCP connections leaving partial ghost files on disk.



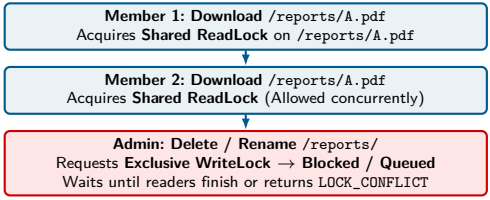
- **Atomic Staging**: Upload writes to hidden temporary file `/storage/.tmp/.upload_UUID.tmp`.
- **Checksum Validation**: Client sends target SHA-256; server verifies byte integrity upon completion.
- **Zero Partial Ghost Files**: Broken connections trigger immediate deletion of the temporary file.

14. Message Processing & Concurrency Pipeline

Server Pipeline Workflow

1. **TCP Demuxer**: Accumulates frames; checks 5B header.
2. **Routing**: Type 0x00 → JSON parser; Type 0x01 → Streamer.
3. **RBAC Guard**: Validates active session and group role.
4. **Lock Arbitration**:
   - `ReadLock`: For `FILE_DOWNLOAD`, `DIR_LIST`.
   - `WriteLock`: For `DIR_RENAME`, `DELETE`, Commit.
5. **Execution & Push**: Executes DB query / file I/O; sends response; broadcasts events to group members.

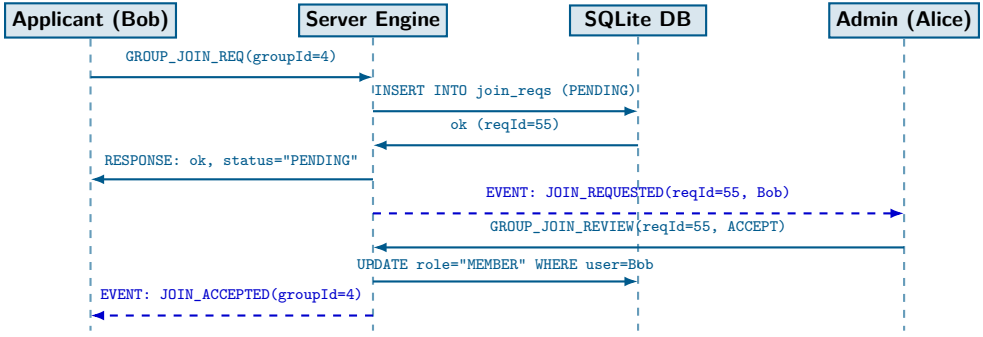
Locking Architecture: Solving Race Conditions



- Eliminates filesystem crashes when admins rename or delete folders while members are actively downloading inside them.

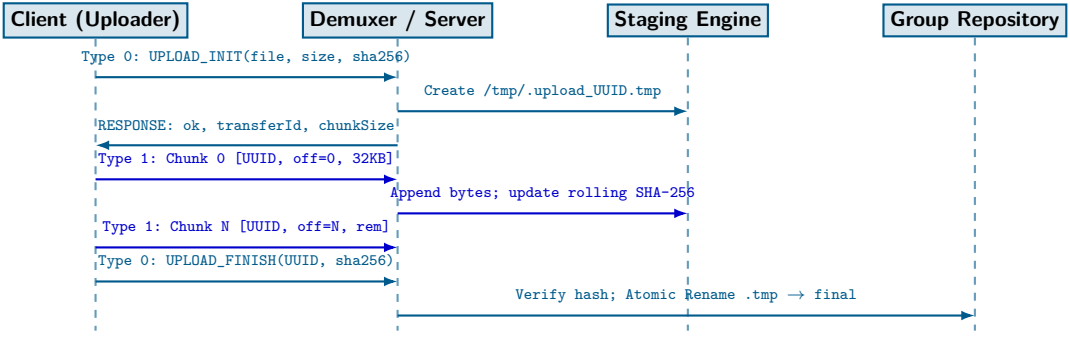
15. Sequence: Group Join Request & Admin Review

Interaction: Applicant Submits Join Request → Admin Approves

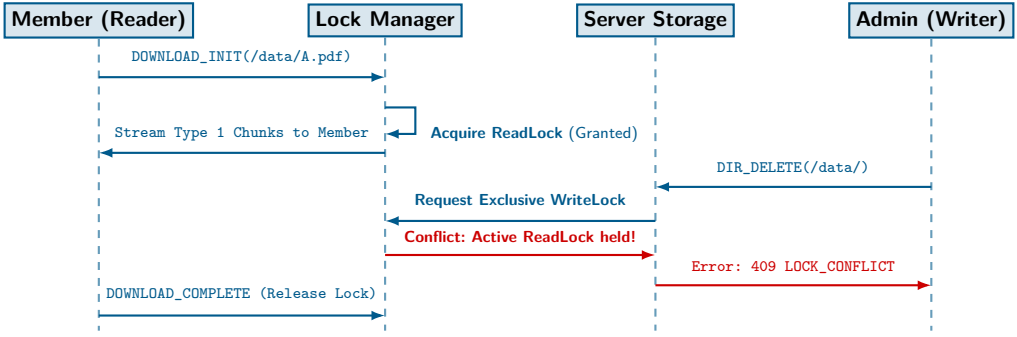


16. Sequence: Chunked Upload & Atomic Commit

Interaction: Chunk Streaming with Integrity Verification & Atomic Move



Conflict Arbitration: Active Download vs Admin Delete Command



18. Implementation Roadmap & Next Milestones

Phase 1 to 3: Core Platform & RBAC

- **Phase 1: Design Review (W1–2)**: Use cases, RBAC matrix, LTV framing, command catalog. (**Status: Done**)
- **Phase 2: Core Socket Engine (W3–4)**: Multithreaded TCP server, byte accumulator, SQLite DB schema, authentication.
- **Phase 3: Group Management (W5–6)**: Group CRUD, join request review, subfolder CRUD, admin delete/rename.

Phase 4 & 5: Streaming & Client GUI

- **Phase 4: Streaming & Staging (W7–8)**:
  - Type 0x01 chunk streaming (32 KiB) & `.tmp` staging.
  - Rolling SHA-256 validation & atomic commit.
  - Fault recovery on socket drop / disconnect.
- **Phase 5: JavaFX GUI & QA (W9–10)**:
  - Desktop client UI with transfer progress bars.
  - Multi-client stress testing & course defense.

19. Summary & Conclusion

Core Contributions in Report 02

- **LTV Framing Protocol**: 5-byte header resolves TCP stream boundary ambiguity cleanly.
- **Multiplexed Streaming**: JSON control frames and raw binary chunks multiplexed over one persistent TCP socket.
- **Rigid RBAC Security**: Group isolation with exclusive admin protection for delete and rename operations.
- **Fault-Tolerant Staging**: Temporary file staging with SHA-256 verification prevents partial ghost files.

Addressed Challenges from Report 01

- **Challenge 1: Concurrent Access**: Path-scoped `ReentrantReadWriteLock` prevents race conditions between readers and writers.
- **Challenge 2: Partial Transfers**: Guaranteed atomic rename from staging file only after full hash verification.
- **Challenge 3: Session State**: Connection-bound session table validates identity without re-sending credentials.



THANK YOU !